From Summation-based PSMT to Encrypted Label Selection and Analytics over Distributed Datasets using FHE

Authors: Nirajan Koirala, Seunghun Paik, Sam Martin, Helena Berens, Tasha Januszewicz, Jonathan Takeshita, Jae Hong Seo, Taeho Jung

NDSS Symposium, 2026

Bikash Thapa

cs.odu.edu/~cs_bthap004

CS800 Assignment 5, Literature Review, Part 2

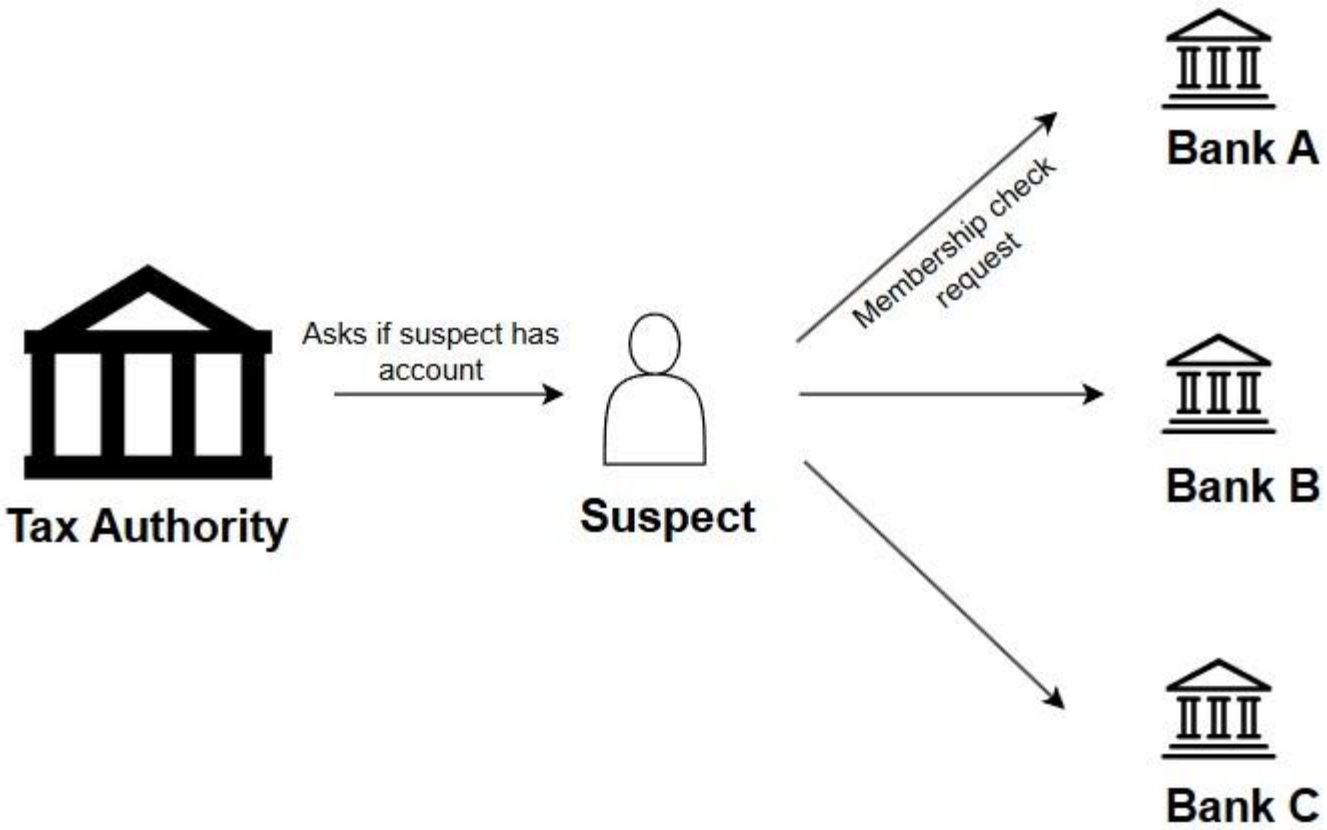
A5, Literature Review, Part 2

Organizations like *banks* and *hospitals* own massive amount of digital data.

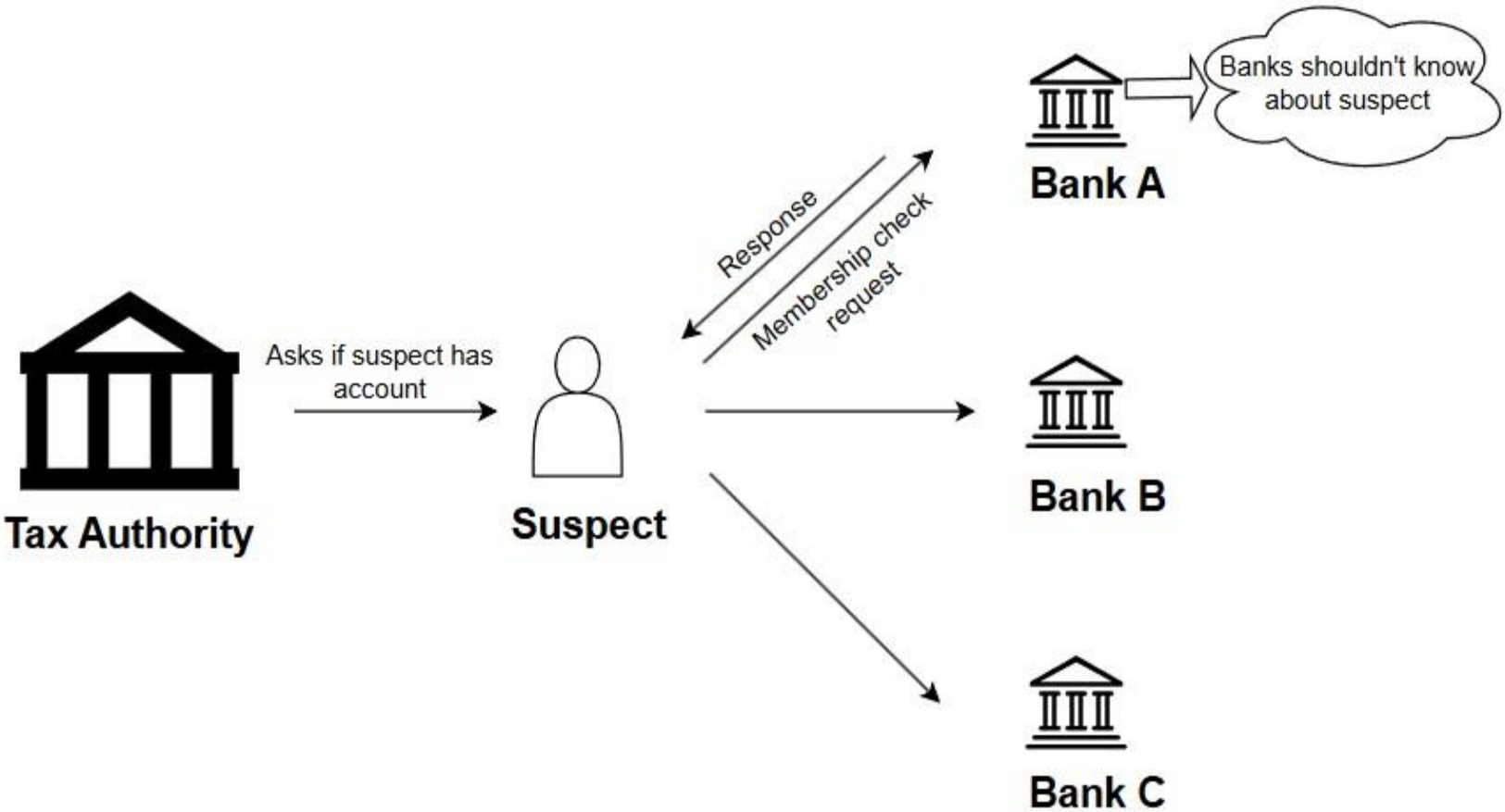
Regulations (GDPR & HIPAA) prohibit from sharing and using these data publicly.

Being able to
share and *compute* on this data *privately* helps in *fraud detection* and *diagnose diseases*.

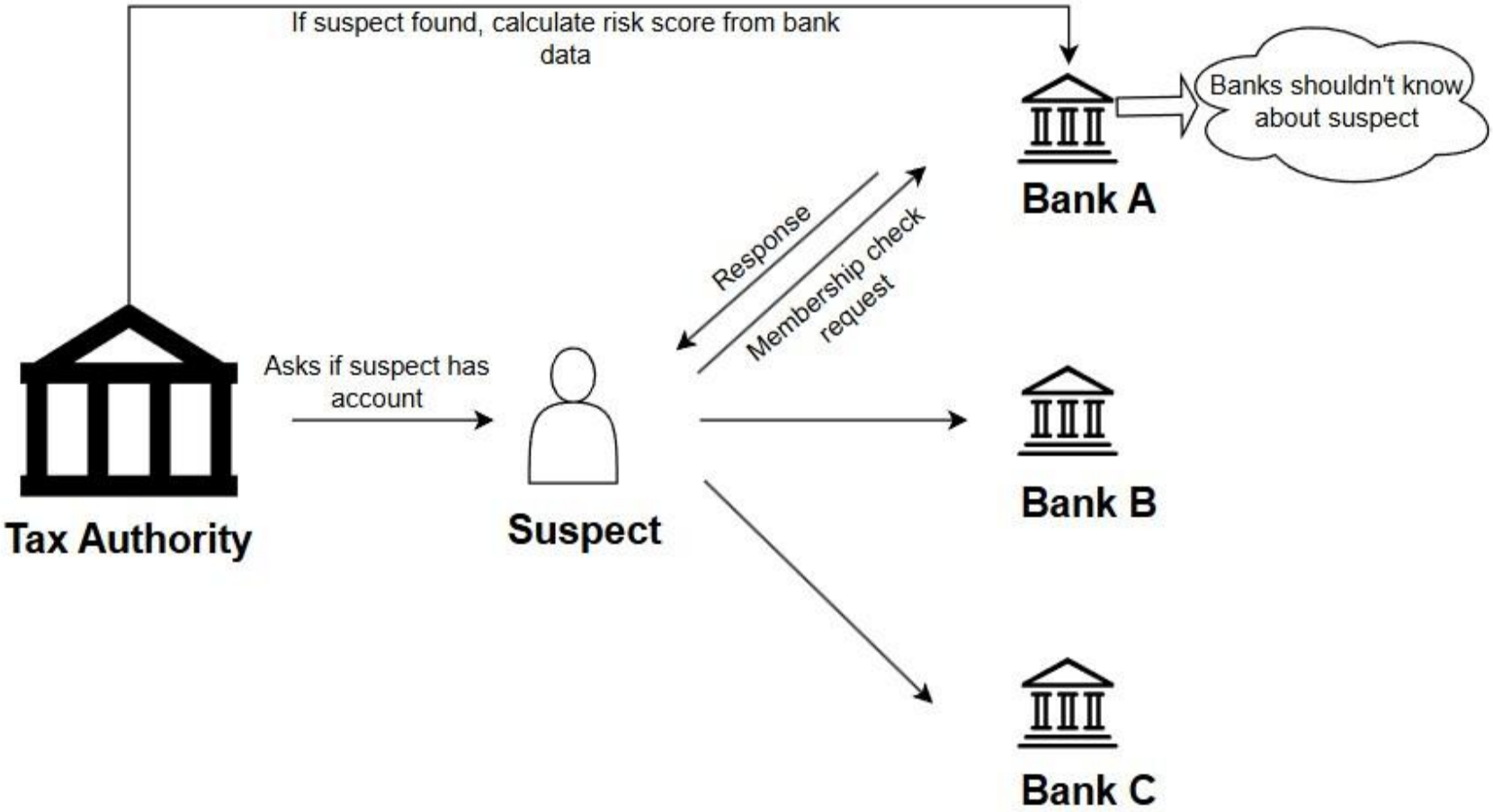
Money Laundering Detection Example - Why private computation required?



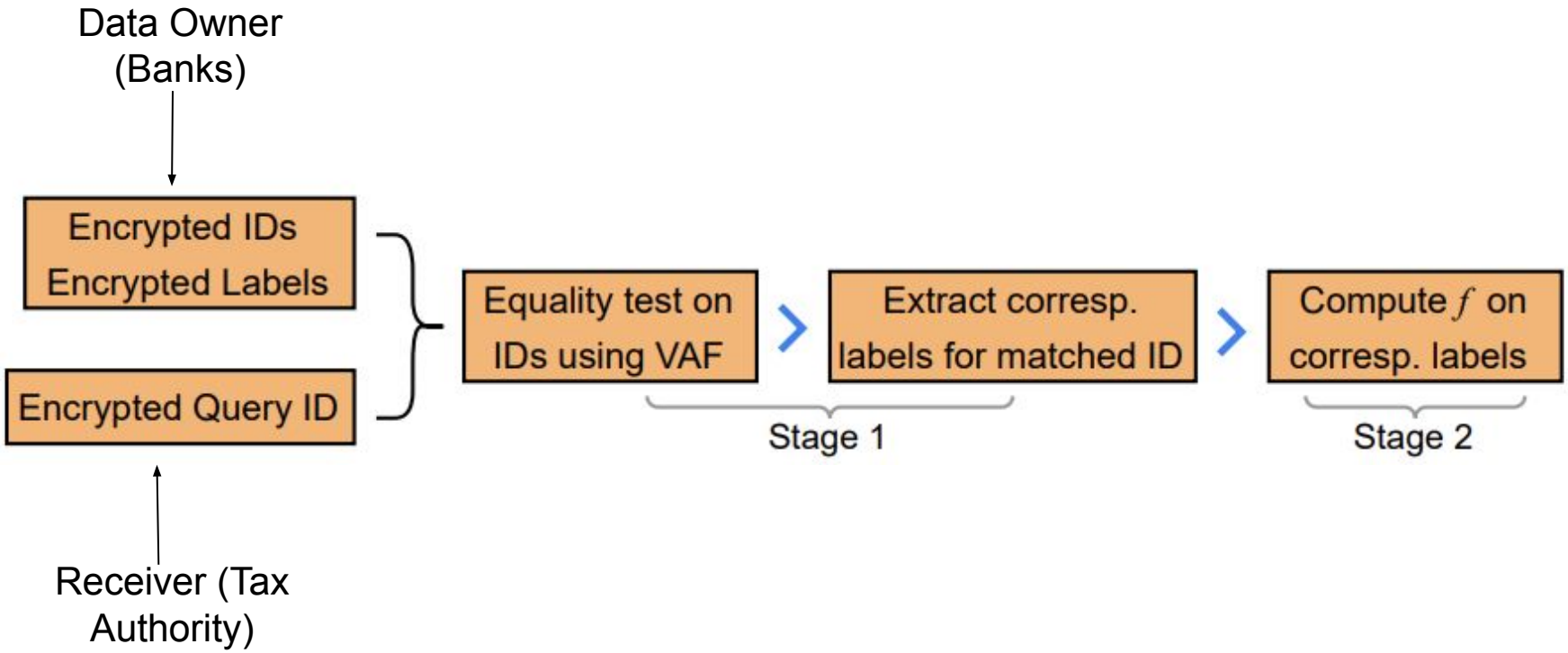
Money Laundering Detection Example - Why private computation required?



Money Laundering Detection Example - Why private computation required?

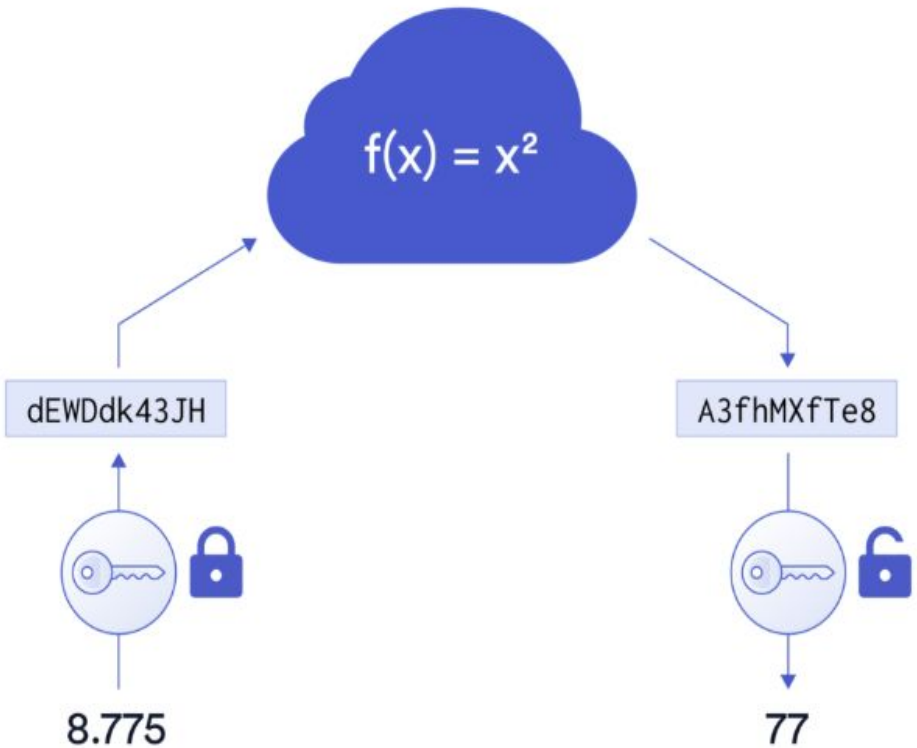


Paper shows that such privacy computation is possible and scalable



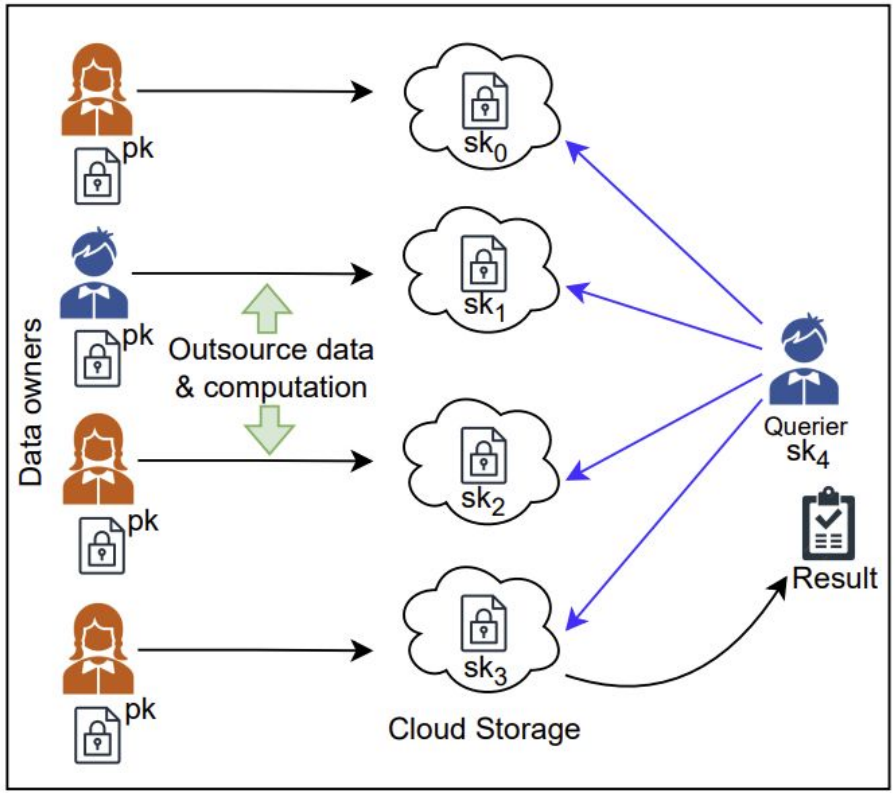
Homomorphic Encryption (HE) allows *addition* and *multiplication* on *encrypted numbers*.

Background - What is Homomorphic Encryption?



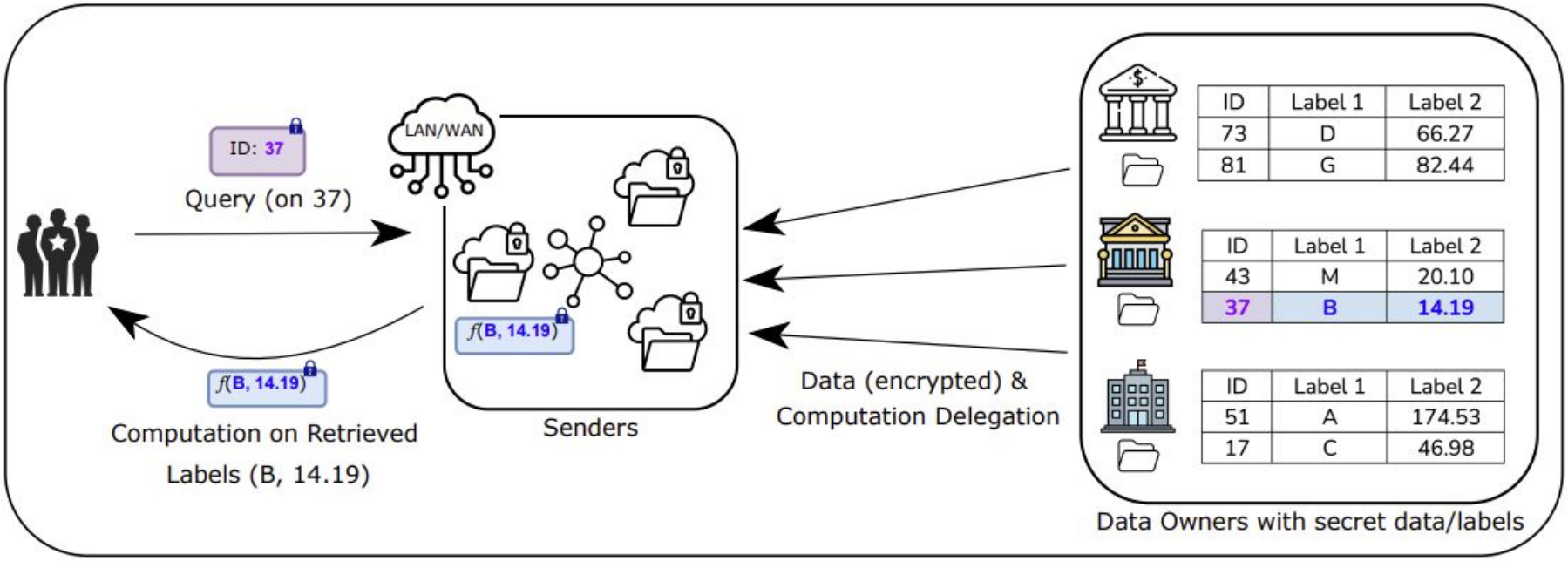
https://chain.link/education-hub/homomorphic-encryption

Background - What is Private Segmented Membership Test ?



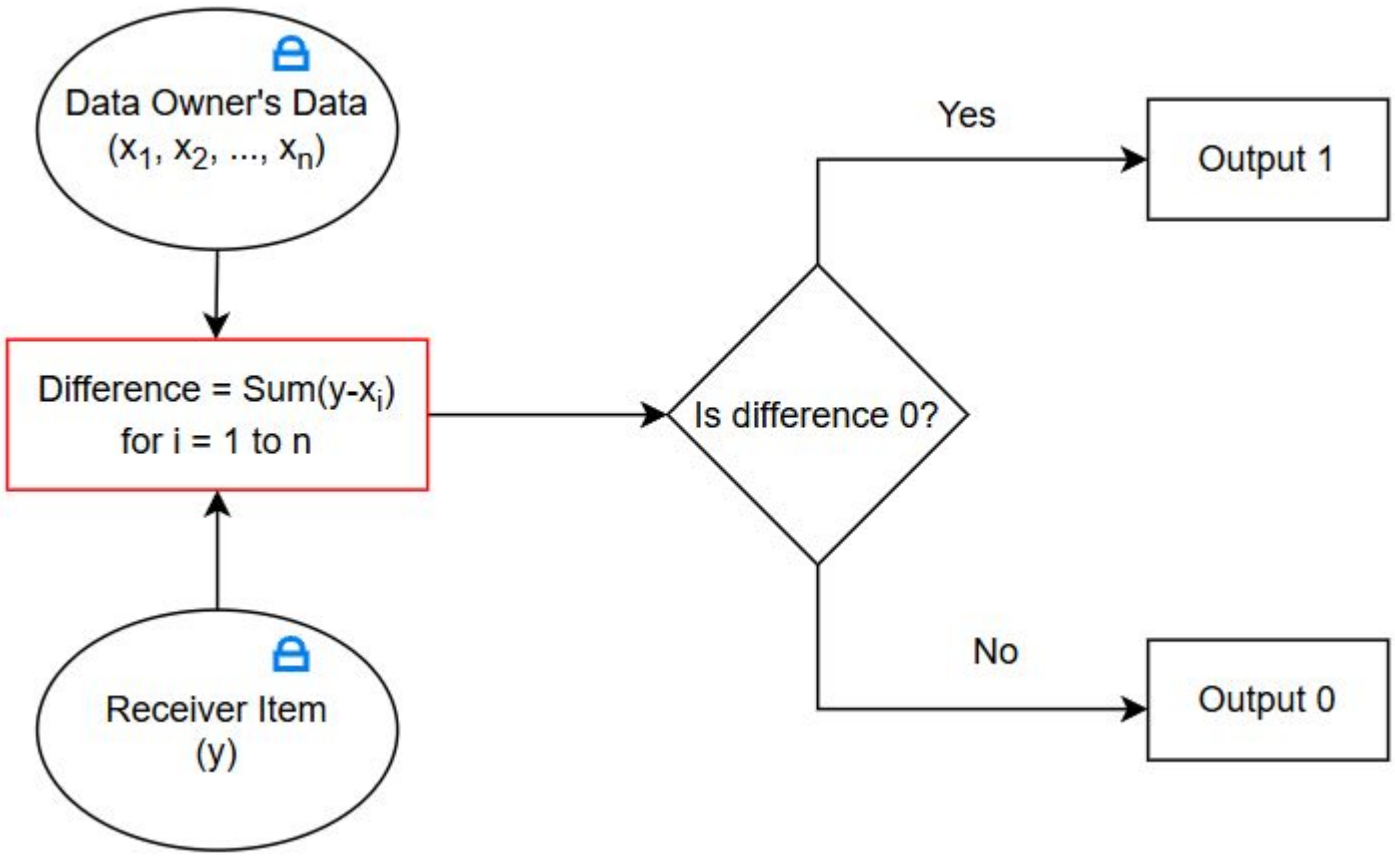
https://doi.org/10.56553/popets-2024-0114

Overview of encrypted label selection and analytics over distributed (data, labels) pairs



- Receiver query an item which ID is 37.
- If ID found on senders, computation is done on matched id labels.
- Receiver receives the match response and computed result (if matched).

Prior Paper Foundation: Summation-based approach to evaluate the membership check



Prior Paper Full Protocol

Parameters: X_i is the input set for l senders: $i \in [0, l-1]$. y, τ, and κ are the receiver's input, threshold value, and limit for the random mask, respectively. λ denotes the computational security parameter, σ denotes the set element's bit string length, and ψ denotes the number of query elements for handling multiple queries. λ, τ, κ, σ, and ψ are public parameters. K and S are public VAF parameters. L, R and n are public DEP parameters. c is the degree of the polynomial used for Chebyshev approximation.

1. **[Parameters]**
   a. **Threshold FHE**: Parties agree on parameters $(N, q, \delta, d, \alpha)$ for the thresFHE CKKS scheme with λ computational security.
   b. **Key Distribution**: Parties run a secure multiparty computation protocol *ThresFHE.KeyGen* distributing distributing shares of the secret key sk among α participants and broadcasting the public key pk and evaluation key evk. Alternatively, a trusted setup can compute these keys. All the parties, including data owners, have access to pk and evk.
2. **[Encryption]**
   a. **Encrypt X & batch** : For all $x \in X_i$, such that $i \in [1, l-1]$, each sender i groups its values x into vectors in $\mathbb{R}^m$ of length m with elements in $\mathbb{R}$. Then, the senders i batches each vector into $2 \cdot m/N$ plaintexts and encrypts them using *ThresFHE.Enc*. Each sender encrypts and batches their sets independently. Alternatively, data owners can execute *ThresFHE.Enc* using pk and send encrypted sets to their corresponding senders i such that $i \in [1, l-1]$. Each sender i obtains a ciphertext c_{x_i} after this step. In case $|X_i| > \eta$, the sender can possess multiple ciphertexts encrypting X_i. Note that FHE batching is applied only on the senders' side unless the receiver needs to transmit multiple queries, in which case batching is also applied on the receiver's side.
   b. **Encrypt $y(s)$**: Receiver constructs a vector of length $N/2$. If using a single query element, each element in the vector is $y \in \mathbb{R}$; otherwise, multiple query elements are inserted into the vector. Then, the receiver encodes the vector into a CKKS plaintext and encrypts it to obtain c_y.
4. **[Compute Intersection]**
   If multiple queries are transmitted by the receiver in a single ciphertext, the sender extracts each query to obtain the ψ number of ciphertexts. Otherwise, the sender receives a single c_y and proceeds without extraction.
   a. **Homomorphically compute subtractions**: Each sender i obtains the ciphertext c_y and computes $\mathit{diff}_i = c_y - c_{x_i}$.
   b. **Apply DEP to reduce the domain size**: Each sender i iteratively applies the domain extension process n times using DEP to shrink the domain interval of values diff_i from step $4a$ into the interval $[-R, R]$.
   c. **Homomorphically evaluate VAF**: Each sender i uses the Chebyshev approximation method to approximate $\mathit{etan}_i = K \cdot (1 - \tanh^2(S \cdot \mathit{diff}_i))$ in the interval $[-R, R]$ using a degree c polynomial.
   d. **First homomorphic squaring**: Each sender i homomorphically squares etan_i and obtains $\mathit{etan}_i^{2^j}$ such that $j \in [1, 12]$. Higher values of $j \geq 2$ can be chosen depending on the desired approximation accuracy for larger senders' set sizes.
   e. **Scaling and second homomorphic squaring**: Each sender i multiplies $\mathit{etan}_i^{2^j}$ using a scaling factor, ρ. Step $4d$ can be applied again to obtain $(\rho \cdot \mathit{etan}_i^{2^j})^{2^k}$ with $k \in [3, 4]$ to exponentially increase the number of parties the protocol can handle and reduce the false positive rates to negligible or zero.
   f. **Homomorphically evaluate summation**: If a sender possesses multiple ciphertexts, Step $4a$ to $4e$ is applied to all the remaining ciphertexts in parallel, and homomorphically summated to $(\rho \cdot \mathit{etan}_i^{2^j})^{2^k}$. An aggregator collects $(\rho \cdot \mathit{etan}_i^{2^j})^{2^k}$ from all other senders, samples a random non-zero plaintext element $r \in [-\kappa, \kappa]$; and homomorphically evaluates z, such that,

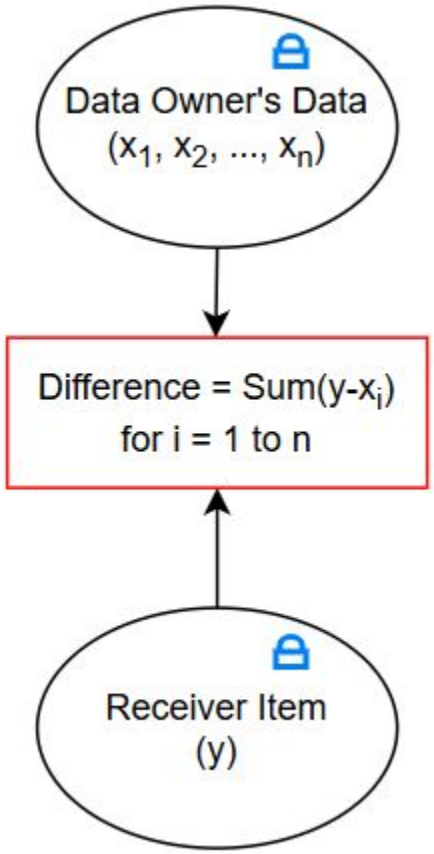
$$z = r + \sum_{i=0}^{l-1} (\rho \cdot \mathit{etan}_i^{2^j})^{2^k}$$

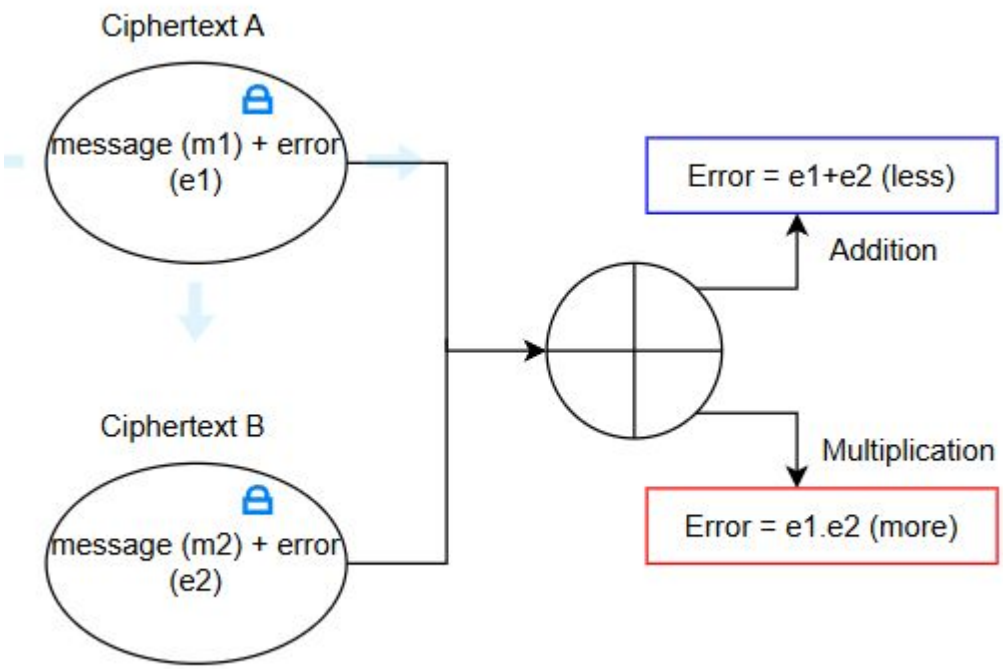
   z is then broadcast to the α parties holding the partial decryption keys, including $\alpha - 1$ senders and the receiver.
5. **[Partial and Final Decryption]**
   a. **Partial decryption & smudging noise** : Upon receiving z, each partial key-share holder sender $i \in [1, \alpha)$ uses their secret key share sk_i to partially decrypt z using *ThresFHE.PartialDec*, which introduces a smudging noise e_{smg} to the partial decryptions $part_i$. Each sender i computes $part_i$ and sends their decryption share to the receiver.
   b. **Final decryption**: The receiver uses their partial decryption key sk_i and z to execute *ThresFHE.PartialDec* and obtains their partial decryption share $part_i$. Optionally, the receiver can use a *ThresFHE.PartialDec* algorithm that does not introduce a smudging noise to increase the precision. Receiver then combines their share with the partial decryptions received from the $\alpha - 1$ senders, $\{part_i : sk_i\}_{i \in [0, \alpha)}$ using *ThresFHE.Combine* and pk and obtains the resulting vector of length η.
   c. **Interpretation of result**: The receiver checks if the magnitude of any element of the resulting vector exceeds τ,

$$\{y\} \cap \bigcup_{i=1}^{l-1} X_i = \{y : \mathit{ThresFHE.Combine}(\mathbf{z}) \geq \tau\}$$

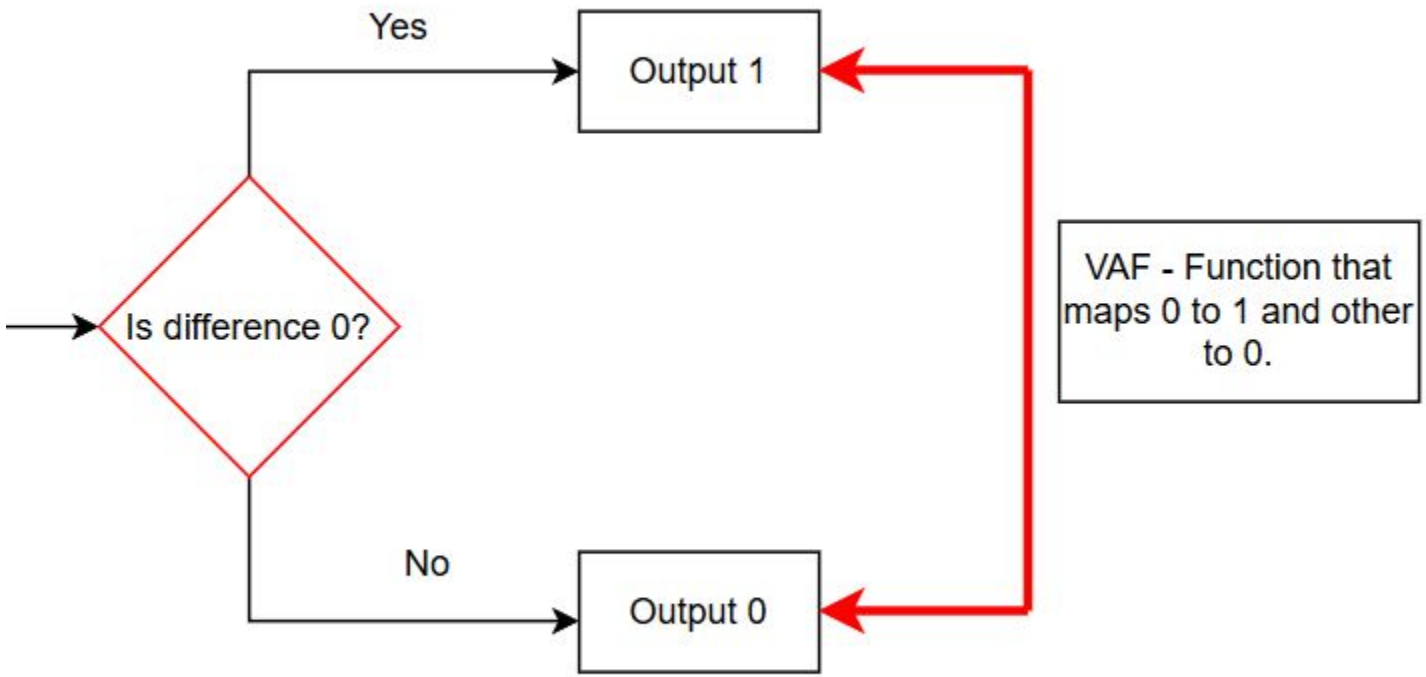
   If any value in a ciphertext slot is greater than the threshold τ, an intersection exists; otherwise, no intersection exists.

Why summation-based approach prioritised over product-based approach?

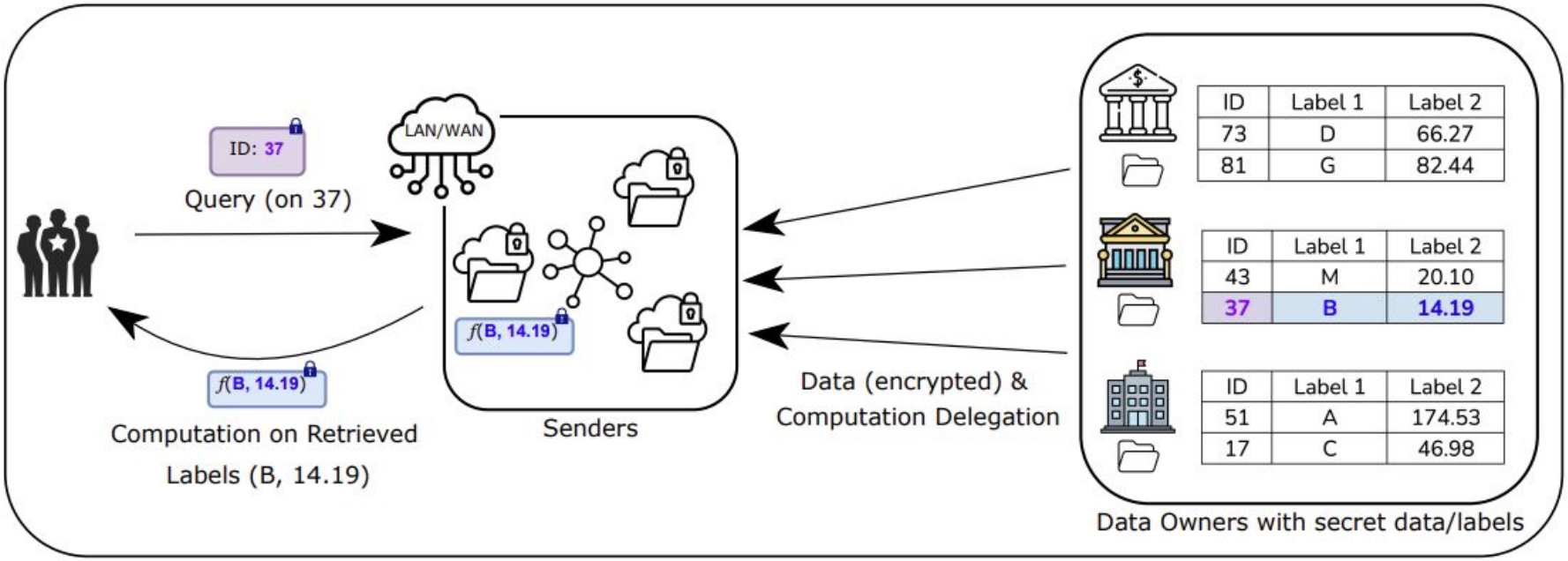




Successive Paper Approach: Summation-based with improved Value Annihilating Function (VAF)



Successive Paper Approach: Function evaluation on Matched Encrypted Labels



- Here, ID 37 is matched. So, its associated labels, Encryption(B,14.9), are extracted.
- Then any arbitrary function f is computed on those encrypted labels.

Successive Paper Full Protocol

Parameters: X_i is the input set for l senders: $i \in [0, l-1]$. y, τ, and κ are the receiver's input, threshold value, and limit for the random mask, respectively. λ denotes the computational security parameter, σ denotes the set element's bit string length, and ψ denotes the number of query elements for handling multiple queries. λ, τ, κ, σ, and ψ are public parameters. K and S are public VAF parameters. L, R and n are public DEP parameters. c is the degree of the polynomial used for Chebyshev approximation.

1. **[Parameters]**
   a. **Threshold FHE**: Parties agree on parameters $(N, q, \delta, d, \alpha)$ for the thresFHE CKKS scheme with λ computational security.
   b. **Key Distribution**: Parties run a secure multiparty computation protocol *ThresFHE.KeyGen* distributing distributing shares of the secret key sk among α participants and broadcasting the public key pk and evaluation key evk. Alternatively, a trusted setup can compute these keys. All the parties, including data owners, have access to pk and evk.
2. **[Encryption]**
   a. **Encrypt X & batch** : For all $x \in X_i$, such that $i \in [1, l-1]$, each sender i groups its values x into vectors in $\mathbb{R}^m$ of length m with elements in $\mathbb{R}$. Then, the senders i batches each vector into $2 \cdot m/N$ plaintexts and encrypts them using *ThresFHE.Enc*. Each sender encrypts and batches their sets independently. Alternatively, data owners can execute *ThresFHE.Enc* using pk and send encrypted sets to their corresponding senders i such that $i \in [1, l-1]$. Each sender i obtains a ciphertext c_{x_i} after this step. In case $|X_i| > \eta$, the sender can possess multiple ciphertexts encrypting X_i. Note that FHE batching is applied only on the senders' side unless the receiver needs to transmit multiple queries, in which case batching is also applied on the receiver's side.
   b. **Encrypt $y(s)$**: Receiver constructs a vector of length $N/2$. If using a single query element, each element in the vector is $y \in \mathbb{R}$; otherwise, multiple query elements are inserted into the vector. Then, the receiver encodes the vector into a CKKS plaintext and encrypts it to obtain c_y.
4. **[Compute Intersection]**
   If multiple queries are transmitted by the receiver in a single ciphertext, the sender extracts each query to obtain the ψ number of ciphertexts. Otherwise, the sender receives a single c_y and proceeds without extraction.
   a. **Homomorphically compute subtractions**: Each sender i obtains the ciphertext c_y and computes $\mathit{diff}_i = c_y - c_{x_i}$.
   b. **Apply DEP to reduce the domain size**: Each sender i iteratively applies the domain extension process n times using DEP to shrink the domain interval of values diff_i from step $4a$ into the interval $[-R, R]$.
   c. **Homomorphically evaluate VAF**: Each sender i uses the Chebyshev approximation method to approximate $\mathit{etan}_i = K \cdot (1 - \tanh^2(S \cdot \mathit{diff}_i))$ in the interval $[-R, R]$ using a degree c polynomial.
   d. **First homomorphic squaring**: Each sender i homomorphically squares etan_i and obtains $\mathit{etan}_i^{2^j}$ such that $j \in [1, 12]$. Higher values of $j \geq 2$ can be chosen depending on the desired approximation accuracy for larger senders' set sizes.
   e. **Scaling and second homomorphic squaring**: Each sender i multiplies $\mathit{etan}_i^{2^j}$ using a scaling factor, ρ. Step $4d$ can be applied again to obtain $(\rho \cdot \mathit{etan}_i^{2^j})^{2^k}$ with $k \in [3, 4]$ to exponentially increase the number of parties the protocol can handle and reduce the false positive rates to negligible or zero.
   f. **Homomorphically evaluate summation**: If a sender possesses multiple ciphertexts, Step $4a$ to $4e$ is applied to all the remaining ciphertexts in parallel, and homomorphically summated to $(\rho \cdot \mathit{etan}_i^{2^j})^{2^k}$. An aggregator collects $(\rho \cdot \mathit{etan}_i^{2^j})^{2^k}$ from all other senders, samples a random non-zero plaintext element $r \in [-\kappa, \kappa]$; and homomorphically evaluates z, such that,

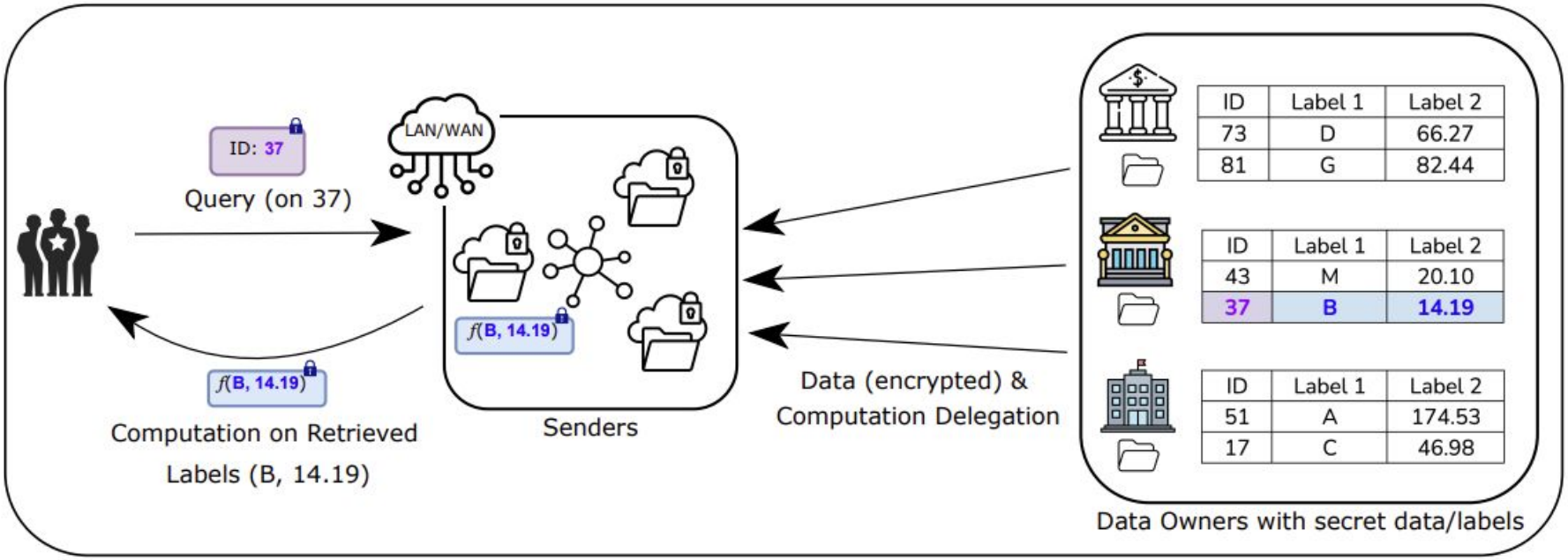
$$z = r + \sum_{i=0}^{l-1} (\rho \cdot \mathit{etan}_i^{2^j})^{2^k}$$

   z is then broadcast to the α parties holding the partial decryption keys, including $\alpha - 1$ senders and the receiver.
5. **[Partial and Final Decryption]**
   a. **Partial decryption & smudging noise** : Upon receiving z, each partial key-share holder sender $i \in [1, \alpha)$ uses their secret key share sk_i to partially decrypt z using *ThresFHE.PartialDec*, which introduces a smudging noise e_{smg} to the partial decryptions $part_i$. Each sender i computes $part_i$ and sends their decryption share to the receiver.
   b. **Final decryption**: The receiver uses their partial decryption key sk_i and z to execute *ThresFHE.PartialDec* and obtains their partial decryption share $part_i$. Optionally, the receiver can use a *ThresFHE.PartialDec* algorithm that does not introduce a smudging noise to increase the precision. Receiver then combines their share with the partial decryptions received from the $\alpha - 1$ senders, $\{part_i : sk_i\}_{i \in [0, \alpha)}$ using *ThresFHE.Combine* and pk and obtains the resulting vector of length η.
   c. **Interpretation of result**: The receiver checks if the magnitude of any element of the resulting vector exceeds τ,

$$\{y\} \cap \bigcup_{i=1}^{l-1} X_i = \{y : \mathit{ThresFHE.Combine}(z) \geq \tau\}$$

   If any value in a ciphertext slot is greater than the threshold τ, an intersection exists; otherwise, no intersection exists.

Let’s walkthrough with our Anti Money Laundering example again



- Here, say ID is account number of the suspect.
- This ID is searched through banks encrypted database.
- If ID matched in database, labels (credit history, transaction behavior) are extracted.
- Risk score is calculated on these labels.
- Encrypted result is sent to tax authority.

Paper approach of *membership check* was *faster* than other state-of-the-art methods.

Evaluation of the developed PSMT Protocol

- Membership check **3.5x to 6.8x faster** than previous **PSMT** and other **state-of-the-art** methods.

- Previous works tested on **100 parties**, while this paper work scaled upto **4096 senders (data owners)**.

- Paper performed experiments on **1.2 million credit transactions (FDB datasets)**, which was completed under **65 seconds**.

Summary and Conclusion

- Paper on **Select Then Compute** adopted and improved previous summation based VAF, solving scalability problem in PSMT.

- Added functionality to compute any arbitrary function on matched ID's labels.

- Experimental results on FDB datasets demonstrated the protocol is fast enough for real-world banking and healthcare use.

Paper 1: https://eprint.iacr.org/2025/072
Paper 2: https://eprint.iacr.org/2025/2023